

Solution to Homework 8

Meshach Aruoriwo Ogbor,
Md Rasel

December 6, 2023

1 Exercise 8.1 - Cartesian Communicators

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 void read_matrix(char *filename, int n_local_rows, int
    n_local_cols, double *matrix_local, int my_rank,
    int q, int rows, int cols);
6 //void circular_shift(int direction, int *coords, int
    *new_coords, int q);
7 //void perform_shifts(int (*shifts)[2], int my_rank,
    int q, MPIComm cart_comm, int rows, int cols, int
    n_local_rows, int n_local_cols, double *
    matrix_local, double *temp_matrix);
8 void write_matrix(char *filename, int n_local_rows,
    int n_local_cols, double *matrix_local, int my_rank
    , int q, int rows, int cols);
9
10 int main(int argc, char **argv) {
11     int my_rank, num_procs;
12     MPIComm cart_comm;
13     int dims[2] = {4, 12};
14     int periods[2] = {1, 1};
15     int coords[2];
16     int shifts_m;
17     int shifts_n;
18     int cart_rank;
```

```

19 FILE *shifts_file;
20 double *matrix, *matrix_local, *temp_matrix;
21
22 MPIComm row_comm;
23 MPIComm col_comm;
24
25 MPI_Init(&argc, &argv);
26 MPI_Comm_rank(MPLCOMM_WORLD, &my_rank);
27 MPI_Comm_size(MPLCOMM_WORLD, &num_procs);
28
29 if (num_procs != dims[0] * dims[1]) {
30     fprintf(stderr, "Number of processes must be 4
31         * 12 = 48\n");
32     MPI_Abort(MPLCOMM_WORLD, 1);
33 }
34 // create communicator with cartesian topology
35 MPI_Cart_create(MPLCOMM_WORLD, 2, dims, periods,
36     1, &cart_comm);
37
38 // get rank with respect to new communicator
39 MPI_Comm_rank(cart_comm, &cart_rank);
40
41 // get position in cartesian process grid
42 MPI_Cart_coords(cart_comm, my_rank, 2, coords);
43
44 if (my_rank == 0) {
45     // Process 0 reads shifts.dat file
46     shifts_file = fopen("/work/korzec/LAB2/ex8/
47         shifts.dat", "r");
48     if (shifts_file == NULL) {
49         fprintf(stderr, "Error opening shifts.dat\n
50             n");
51         MPI_Abort(MPLCOMM_WORLD, 1);
52     }
53
54     //for (int i = 0; i < 1000; i++) {
55         //fscanf(shifts_file, "%d %d", &shifts[i
56             ][0], &shifts[i][1]);
57         fscanf(shifts_file, "%d", &shifts_m);
58         fscanf(shifts_file, "%d", &shifts_n);

```

```

54         fclose(shifts_file);
55     }
56
57     // Broadcast shifts to all processes
58     MPI_Bcast(&shifts_m, 1, MPI_INT, 0, MPLCOMM_WORLD
59             );
60     MPI_Bcast(&shifts_n, 1, MPI_INT, 0, MPLCOMM_WORLD
61             );
62
63     // Read matrix from file
64     int rows = 400, cols = 600;
65     int n_local_rows = rows / dims[0];
66     int n_local_cols = cols / dims[1];
67     matrix_local = (double *)malloc(n_local_rows *
68                                     n_local_cols * sizeof(double));
69     temp_matrix = (double *)malloc(n_local_rows *
70                                    n_local_cols * sizeof(double));
71
72     read_matrix("/work/korzec/LAB2/ex8/notstirred.txt",
73                n_local_rows, n_local_cols, matrix_local,
74                cart_rank, dims[0], rows, cols);
75
76     // Perform circular shifts based on shifts.dat
77     //perform_shifts(shifts, my_rank, dims[0],
78                     cart_comm, rows, cols, n_local_rows,
79                     n_local_cols, matrix_local, temp_matrix);
80     MPI_Cart_sub(cart_comm, (int [2]) {0,1}, &row_comm
81             );
82     MPI_Cart_sub(cart_comm, (int [2]) {1,0}, &col_comm
83             );
84
85     // Write the final matrix to file
86     write_matrix("transformed_matrix.txt",
87                 n_local_rows, n_local_cols, matrix_local,
88                 my_rank, dims[0], rows, cols);
89
90     free(matrix_local);
91     free(temp_matrix);
92
93

```

```

82     MPI_Finalize();
83
84     return 0;
85 }
86
87 void read_matrix(char *filename, int n_local_rows, int
      n_local_cols, double *matrix_local, int my_rank,
88 int q, int rows, int cols) {
89     if (my_rank == 0) {
90         FILE *file = fopen(filename, "r");
91         if (file == NULL) {
92             fprintf(stderr, "Error opening %s\n",
93                     filename);
94             MPI_Abort(MPLCOMM_WORLD, 1);
95         }
96
97         int n = rows * cols;
98         double *matrix = (double *)malloc(n * sizeof(
99             double));
100
101         for (int i = 0; i < n; i++) {
102             fscanf(file, "%lf", &matrix[i]);
103         }
104
105         fclose(file);
106
107         MPI_Scatter(matrix, n_local_rows *
108             n_local_cols, MPLDOUBLE, matrix_local,
109             n_local_rows * n_local_cols, MPLDOUBLE, 0,
110             MPLCOMM_WORLD);
111
112         free(matrix);
113     } else {
114         MPI_Scatter(NULL, 0, MPLDATATYPE_NULL,
115             matrix_local, n_local_rows * n_local_cols,
116             MPLDOUBLE, 0, MPLCOMM_WORLD);
117     }
118 }
119
120 /*void circular_shift(int direction, int *coords, int

```

```

113     *new_coords, int q) {
114         switch (direction) {
115             case 0:
116                 new_coords[0] = (coords[0] + 1) % q;
117                 new_coords[1] = coords[1];
118                 break;
119             case 1:
120                 new_coords[0] = (coords[0] + q - 1) % q;
121                 new_coords[1] = coords[1];
122                 break;
123             case 2:
124                 new_coords[0] = coords[0];
125                 new_coords[1] = (coords[1] + 1) % 12;
126                 break;
127             case 3:
128                 new_coords[0] = coords[0];
129                 new_coords[1] = (coords[1] + 11) % 12;
130                 break;
131             default:
132                 break;
133         }
134     }
135     /*
136     void perform_shifts(int (*shifts)[2], int my_rank,
137                         int q, MPIComm cart_comm, int rows, int cols, int
138                         n_local_rows, int n_local_cols, double *
139                         matrix_local, double *temp_matrix) {
140         int coords[2], new_coords[2];
141         MPI_Cart_coords(cart_comm, my_rank, 2, coords);
142
143         for (int i = 0; i < 1000; i++) {
144             circular_shift(shifts[i][0], coords,
145                             new_coords, q);
146
147             int dest_rank;
148             MPI_Cart_rank(cart_comm, new_coords, &
149                             dest_rank);
150
151             MPI_Sendrecv(matrix_local, n_local_rows *
152                             n_local_cols, MPLDOUBLE, dest_rank, 0,

```

```

        temp_matrix, n_local_rows * n_local_cols,
        MPI.DOUBLE, dest_rank, 0, cart_comm,
        MPI_STATUS_IGNORE);
146
147     for (int j = 0; j < n_local_rows *
        n_local_cols; j++) {
148         matrix_local[j] = temp_matrix[j];
149     }
150 }
151 }
152 */
153 void write_matrix(char *filename, int n_local_rows,
        int n_local_cols, double *matrix_local, int my_rank
        , int q, int rows, int cols) {
154     if (my_rank == 0) {
155         FILE *file = fopen(filename, "w");
156         if (file == NULL) {
157             fprintf(stderr, "Error opening %s\n",
                filename);
158             MPI_Abort(MPI_COMM_WORLD, 1);
159         }
160
161         int n = rows * cols;
162         double *matrix = (double *)malloc(n * sizeof(
            double));
163
164         MPI_Gather(matrix_local, n_local_rows *
            n_local_cols, MPI.DOUBLE, matrix,
            n_local_rows * n_local_cols, MPI.DOUBLE, 0,
            MPI_COMM_WORLD);
165
166         fprintf(file, "%d %d\n", rows, cols);
167         for (int i = 0; i < n; i++) {
168             fprintf(file, "%lf\n", matrix[i]);
169         }
170
171         fclose(file);
172
173         free(matrix);
174     } else {

```

```
175         MPI_Gather(matrix_local, n_local_rows *  
                    n_local_cols, MPLDOUBLE, NULL, 0,  
                    MPLDATATYPE_NULL, 0, MPLCOMM_WORLD);  
176     }  
177 }
```

For compilation on Stromboli, we used the following command:
`mpicc -o ex8 ex8.c`

2 Submit Script for exercise 8.1

```
1 #!/bin/bash
2 #SBATCH --nodes=3
3 #SBATCH --ntasks=48
4 #SBATCH --partition=compute2011
5 #SBATCH --exclusive
6 ##SBATCH --output=output_ex8.txt
7
8 module load mpi/openmpi/4.1.0
9 mpirun ./ex8
```

3 Results

We have faced some difficulties while reading the `shifts.dat` file. At first, we tried to do it in a loop, but at the end we did it without a for loop. Secondly, we could not correctly exchange values among the neighbour ranks. Due to that, our final matrix might not be the correct one, which you can verify from the image we got from the final matrix. We will appreciate if you can guide us how to fix this.

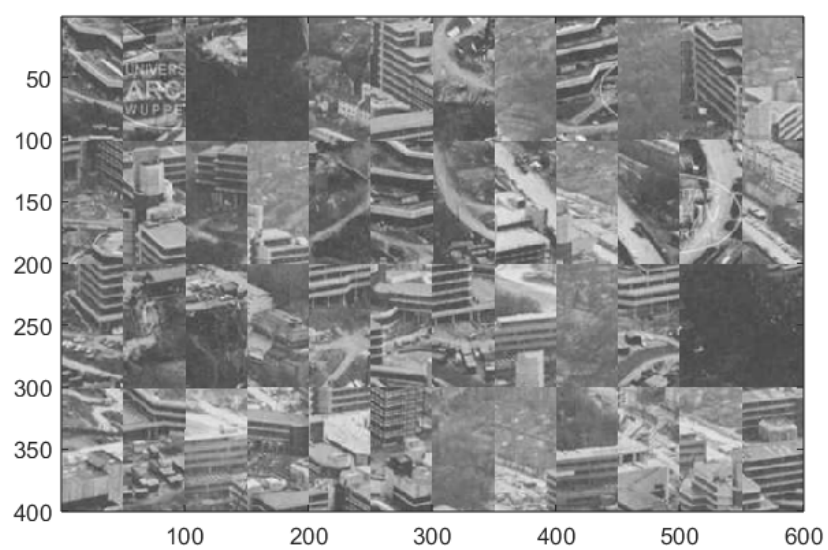


Figure 1: Image from original matrix

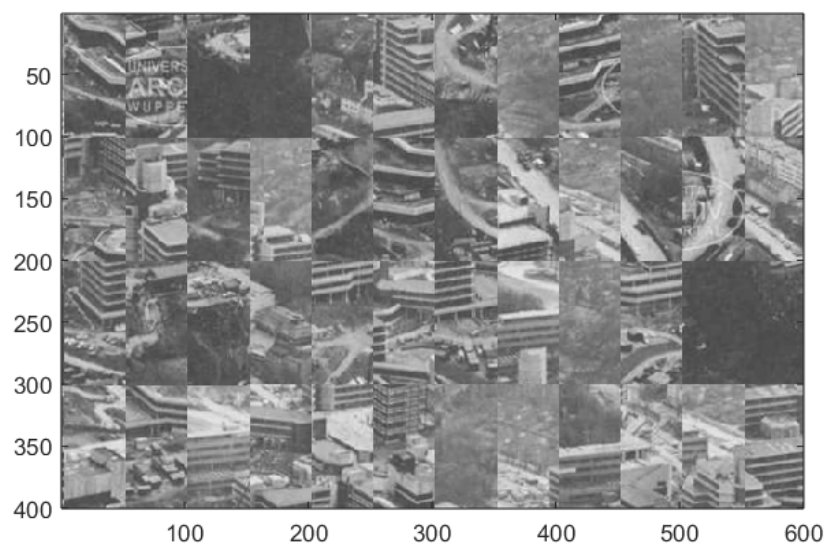


Figure 2: Image from final matrix